

Software Requirements Specification (SRS)

AURA: Modern Artisan Kitchen & Botanica

Frontend MVP · React Native + Expo SDK

Document Version: 2.0.0

Academic Term: Fall 2026

Course: Mobile Application Development (CS-3112 / SE-3112)

Deliverable: Question 1: Comprehensive System Specification

Executive Summary & Client Interview

To ground the requirements in real-world human experience, a preliminary client self-interview was conducted from both Customer and Restaurant Manager perspectives:

- **The Customer Persona (Zainab Malik, VIP Dining Patron, Lahore):** *I frequently visit high-end artisan bistros during dinner rushes and special occasions. When planning an evening out, I require instant access to the latest seasonal menu, ingredient details, real-time table booking with immediate seat allocation, and the ability to customize my orders in advance to avoid waiting lines.*
- **The Manager Persona (Chef Farhan Siddiqui, Executive Head Chef & Operations GM):** *During peak kitchen hours, phone bookings and paper order tickets result in lost notes, table double-bookings, and service delays. We need a live, digitized operational console to supervise incoming kitchen tickets, approve or decline table reservations dynamically, and update dish pricing and availability instantly across all diner screens.*

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) details the complete functional and non-functional requirements for the **Restaurant App MVP (AURA: Modern Artisan Kitchen & Botanica)**. The target audience includes academic evaluators, mobile software engineers, UI/UX designers, and quality assurance testers. This specification strictly reflects a frontend-only architecture powered by React Native and Expo SDK, utilizing local state machines and simulated persistence without a physical backend server.

1.2. Scope

In-Scope Functionalities:

1. **Authentication & Role Authorization:** Dual-mode authentication (Customer & Manager) utilizing mock user registries and session storage. Powered by the custom `useForm` hook with instant field validation, 3D flip card animations, and one-touch VIP RFID keycard credentials ([Zainab Malik](#) & [Chef Farhan Siddiqui](#)).
2. **Interactive Menu Catalog:** Categorized menu browsing (Starters, Mains, Desserts, Drinks) featuring 16 signature culinary dishes priced in Pakistani Rupees (Rs.), simulated network delay (1.5s), pull-to-refresh, dynamic item counting, and "100% Halal Artisan" / "Daily Special" badges.

3. **Optimized Search & Filter Controls:** Real-time text search powered by a custom `useDebounce` hook (400ms delay), recent search suggestion chips, horizontal category filtering, multi-criteria sorting (Price, Name, Recommended), and a 300px threshold "Back to top" floating button.
4. **Adaptive Luxury Theming:** System-wide dark and light mode toggle using custom React Context (`ThemeContext`), instantly transitioning between an **Obsidian Midnight** palette and a **Sunlit Botanical Alabaster** palette.
5. **Cart & Order Customization:** Pure reducer-driven cart (`cartReducer.js`) supporting all 8 actions: quantity increments/decrements, item removal, dietary instructions, promotional voucher discounts (`AURA10`, `AURA20`, `AURA50`), and real-time tab badge indicators.
6. **Financial Calculation & Performance Optimization:** Subtotal, service charges (5%), and sales tax (15%) calculated via memoized formulas (`useMemo`); food card re-renders optimized via `React.memo` and stable `useCallback` action references.
7. **Table Reservation System:** Custom hook-based booking pipeline (`useReservation.js`) featuring slot conflict detection (12:00 to 22:00), party capacity checks (1-12 guests), Pakistani phone format validation (`03XX-XXXXXXX`), and reservation cancellations.
8. **Live Order Progression & Manager Operations:** Dine-in vs. Takeaway checkout, timed status progression (10s Preparing → 20s Ready → 30s Served), live elapsed timer, and a dedicated 3-tab Manager Dashboard.
9. **Black-Tie VIP Patron Passport:** Dedicated patron profile screen featuring gold-foiled digital membership cards, dining analytics matrix (Visits, Spend, Points), concierge dining preferences, and secure session sign out.

Out-of-Scope (Excluded from MVP):

1. **Live Payment Gateway:** No real credit card or third-party payment processing (Stripe, JazzCash, EasyPaisa).
2. **Native Push Notifications:** Apple APNs or Google FCM remote push notification daemons are excluded.
3. **Remote Backend & Persistent Relational Database:** No remote REST/GraphQL microservices, Node.js server, Firebase, or cloud SQL databases.
4. **Hardware Geolocation / GPS Tracking:** Real-time courier dispatch and GPS map rendering are excluded.

1.3. Definitions, Acronyms, and Abbreviations

1. **SRS (Software Requirements Specification):** A structured document outlining functional capabilities, constraints, and architecture of a software product.
2. **MVP (Minimum Viable Product):** A version of a software product with sufficient core features to validate requirements and usability.
3. **UML (Unified Modeling Language):** Standardized visual modeling language used to visualize, specify, and document software system artifacts.
4. **React Hook:** Special JavaScript functions (e.g., `useState`, `useEffect`, `useRef`, `useMemo`, `useCallback`, `useReducer`) enabling functional components to manage state and lifecycle events.
5. **Context API:** React engine feature facilitating global state propagation across deeply nested component trees without manual prop drilling.
6. **Pure Reducer:** A deterministic function (`state, action`) => `newState` that calculates subsequent application state without mutating original inputs or causing side effects.

7. **Mock Data:** Localized JSON or JavaScript data structures simulating realistic database entities for standalone frontend development.
8. **FlatList:** A memory-efficient React Native component that virtualizes and renders only currently visible items within large scrollable datasets.
9. **Debounce:** A software design technique that restricts the execution of a computationally expensive task until a specified duration of inactivity has elapsed.
10. **Immutable State:** An architectural paradigm where data objects cannot be modified post-creation; state transitions always yield fresh object references.

2. Overall Description

2.1. User Roles & Personas

Role	Description	Key Permissions & Goals
Customer (Zainab Malik, VIP Patron)	Registered dining patron seeking luxury	• Browse menu, view daily specials, and order tracking. • Search with 400ms debouncing • Customize cart with chef instructions • Apply promo vouchers (AURA10, AURA20, AURA50) • Book/cancel table reservations • Place Dine-in/Takeaway orders • Toggle dark/light themes
Restaurant Manager (Chef Farhan Siddiqui)	Administrative culinary supervisor overseeing	• Manage operations, staff, and inventory. • Monitor & advance kitchen orders • Review/accept/decline reservations • Update prices & availability • Create new menu items

2.2. User Stories

Customer User Stories (Derived from Client Meeting / Self-Interview):

1. **US-01 (Menu Discovery):** *As a Customer*, I want to view dishes categorized into Starters, Mains, Desserts, and Drinks with appetizing images and prices in PKR.
2. **US-02 (Daily Specials & Halal Certification):** *As a Customer*, I want chef specials and 100% Halal artisan certifications clearly highlighted with distinctive badges.
3. **US-03 (Instant Debounced Search):** *As a Customer*, I want a fast search bar powered by `useDebounce` with recent search history chips.
4. **US-04 (Dietary Customization):** *As a Customer*, I want to attach per-item special instructions to dishes in my cart.
5. **US-05 (Promotional Vouchers):** *As a Customer*, I want to enter promo codes (`AURA10`, `AURA20`) during checkout for discounts.
6. **US-06 (Advance Table Booking):** *As a Customer*, I want to reserve a table for a specific date, hourly slot, and party size with automated capacity validation.
7. **US-07 (Live Order Progress):** *As a Customer*, I want to track my order through an automated progress timeline with elapsed seconds.

Restaurant Manager User Stories:

8. **US-08 (Kitchen Queue Progression):** *As a Restaurant Manager*, I want a real-time incoming orders dashboard where I can advance kitchen tickets.
9. **US-09 (Reservation Management):** *As a Restaurant Manager*, I want to review incoming dining reservations and explicitly Accept or Decline them.
10. **US-10 (Live Menu CRUD & Availability):** *As a Restaurant Manager*, I want to adjust menu item prices and toggle out-of-stock items off in real-time.
11. **US-11 (Role-Restricted Console):** *As a Restaurant Manager*, I want management tabs completely hidden from ordinary diner profiles.
12. **US-12 (Theme Personalization):** *As a Customer or Manager*, I want to toggle between obsidian dark and porcelain emerald light themes.

3. Functional Requirements

Module 1: Authentication & Authorization

- **FR-01:** The system shall toggle between Login and Signup modes with 3D flip card animations.
- **FR-02:** The system shall manage all form state through a custom `useForm` hook (`values`, `errors`, `handleChange`, `handleSubmit`, `reset`, `isValid`).
- **FR-03:** The system shall clear field-specific error messages automatically when the user modifies text.
- **FR-04:** The system shall authenticate credentials against a local mock user registry with one-touch VIP RFID digital keycards (`Zainab Malik`: Patron, `Chef Farhan Siddiqui`: Manager).
- **FR-05:** The system shall route Customer users to Menu and Manager users to Manager Dashboard.

Module 2: Menu Browsing & Catalog

- **FR-06:** The system shall asynchronously load 16 artisan culinary items spanning 4 categories using a simulated 1.5-second Promise with unmount cancellation.
- **FR-07:** The system shall present horizontal category filtering chips for specific course or all items.
- **FR-08:** The system shall render catalog items in an optimized `FlatList`, displaying "Daily Special" badges and greying out unavailable dishes.
- **FR-09:** Individual dish cards shall be memoized using `React.memo` and stable `useCallback` references.

Module 3: Search, Filter & Scroll Optimization

- **FR-10:** The system shall provide a search bar with persistent `useRef` focus trigger and 1-tap clear button.
- **FR-11:** The system shall debounce search queries by 400ms via the custom `useDebounce` hook.
- **FR-12:** The system shall present the last five unique search queries as tappable suggestions.
- **FR-13:** The system shall display a floating "Back to top" button when scrolled past 300px.
- **FR-14:** The system shall compute search results, category filtering, and multi-criteria sorting in a single `useMemo` expression.

Module 4: Shopping Cart & Reducer State

- **FR-15:** The system shall manage cart operations through a pure `cartReducer` handling 8 distinct action types: `ADD_ITEM`, `REMOVE_ITEM`, `INCREMENT_QUANTITY`, `DECREMENT_QUANTITY`, `CLEAR_CART`, `APPLY_PROMO`, `REMOVE_PROMO`,

and `SET_ORDER_TYPE`.

- **FR-16:** The system shall validate promotional vouchers (`AURA10` for 10%, `AURA20` for 20%, `AURA50` for 50%).
- **FR-17:** The system shall update a live badge indicator on the bottom navigation tab icon.

Module 5: Order Summary & Financial Calculation

- **FR-18:** The system shall compute Subtotal, Service Charge (5%), Provincial Sales Tax (15%), Promotional Discounts, and Grand Total using memoized expressions.
- **FR-19:** The system shall support both Dine-in (with table assignment) and Takeaway (with pickup time) modes.

Module 6: Table Reservation Pipeline

- **FR-20:** The system shall provide hourly reservation slots between 12:00 and 22:00.
- **FR-21:** The system shall validate reservation parameters via `useReservation` hook (past date check, party size 1-12, Pakistani mobile formatting `03XX-XXXXXX`).
- **FR-22:** The system shall present a booking confirmation modal and permit cancellations.

Module 7: Live Order Tracking & Manager Dashboard

- **FR-23:** The system shall auto-transition order status: `Pending` → `Preparing` (10s) → `Ready` (20s) → `Served` (30s) with live elapsed counter.
- **FR-24:** The system shall provide a Manager Dashboard with 3 operational views: (a) Incoming Orders, (b) Reservation approvals/declines, (c) Live Menu Management.
- **FR-25:** The system shall synchronize menu modifications and order transitions instantly via shared Context providers and `@aura_*` storage keys.

Module 8: Patron Profile & Concierge Lounge

- **FR-26:** The system shall render a luxury Black-Tie VIP Patron Passport Card with membership tier (`EMERALD PRIVILEGE PATRON`) and member ID.
- **FR-27:** The system shall display dining metrics pods (visits, spend in PKR, reward points).
- **FR-28:** The system shall offer concierge dining preference selectors and illuminated dark/light theme switch.
- **FR-29:** The system shall provide secure session sign out that wipes tokens and resets navigation.

4. Non-Functional Requirements

- **NFR-01 (Usability & Aesthetics):** Luxury dining design aesthetics with emerald teal (`#0D9488`), radiant mint (`#14B8A6`), champagne gold (`#D97706`), and obsidian slate (`#0F172A`) color palettes, 16px touch targets, smooth 3D button animations.
- **NFR-02 (Performance & 60 FPS Scrolling):** `MenuItemCard` memoized with `React.memo` and stable `useCallback` references.
- **NFR-03 (Responsiveness):** Layouts adapt across 360px to 480px viewports and tablet form factors.
- **NFR-04 (Maintainability):** Modular separation under `src/` (components, screens, context, reducers, hooks, data, navigation, theme).

- **NFR-05 (Client-Side Data Integrity):** No external server dependencies; persistence via `safeStorage` (in-memory + `AsyncStorage` fallback with `@aura_*` keys).
- **NFR-06 (Memory Leak Prevention):** Explicit cleanup for all `setTimeout` / `setInterval` on component unmount.

5. Client-Side Data Model (Mock Data Schema)

Data Set	Primary Fields	Business Purpose
Users	id, name, email, password, role, phone, membership	Stores patron/staff profiles (Zainab Malik, Chef Farhan Siddiqui, Hamza Abbasi) for authentication.
Categories	id, name, icon	Enumerates 4 culinary courses: Starters, Mains, Desserts, Drinks.
MenuItems	id, name, description, price, category, image, isSpecial, availability	Details of food & beverages, editable by kitchen managers.
Tables	id, number, capacity, location, isOccupied	Physical dining tables for seating assignment & party matching.
Reservations	id, customerName, customerPhone, partySize, date, timeSlot	Table bookings reserved, created & managed by managers.
Orders	id, userId, customerName, items, subtotal, serviceCharge, tax, total, status, timestamp	Change, deliver, order, tracked live by waiters & staff.

6. UML Architectural Diagrams & Summaries

6.1. Use Case Diagram

- **Caption:** Figure 6.1: AURA Use Case Diagram (Actors: Customer & Restaurant Manager).
- **Architectural Explanation:** The use case diagram models interactions between the primary actors (Customer and Restaurant Manager) and the system boundaries. The Customer interacts with 8 core use cases. The Place Order use case demonstrates include/extend relationships with Calculate Totals and Apply Promo Code.

6.2. Class / Domain Model Diagram

- **Caption:** Figure 6.2: Object-Oriented Domain Model & Class Relationships.
- **Architectural Explanation:** The domain model represents system entities and structural multiplicities. User is specialized into Customer and Manager. A Customer aggregates one Cart with 0..* CartItem instances referencing MenuItem objects. Orders are generated from Carts with optional Table references.

6.3. Sequence Diagram

- **Caption:** Figure 6.3: End-to-End Sequence Diagram for Order Placement.
- **Architectural Explanation:** Traces chronological message exchanges from MenuScreen through cartReducer, OrdersContext, to OrderTrackingScreen with interval-based status progression.

6.4. State Machine Diagram

- **Caption:** Figure 6.4: Finite State Machine for Culinary Order Lifecycle.
- **Architectural Explanation:** States: Pending → Preparing → Ready → Served, with Cancelled branch from Pending. Transitions by automated timers (10s, 20s, 30s) or manual manager overrides.

6.5. Component Diagram

- **Caption:** Figure 6.5: React Native Component & Context Dependency Diagram.

- **Architectural Explanation:** Root AppNavigator wraps SafeAreaProvider and 5 Context Providers (ThemeProvider, AuthProvider, MenuProvider, CartProvider, OrdersProvider). Screens consume via custom hooks (useTheme, useAuth, useCart, useOrders, useReservation, useForm, useDebounce).

7. MVP Frontend Development (Screen Specifications)

7.1. Login and Signup Screen (`LoginScreen.js`)

- **Purpose:** Single-screen authentication portal featuring 3D flip card animations, architectural crest medallion, sliding segmented floor pass, and twin VIP RFID keycard credentials.
- **UI Elements:** Sliding floor switcher tabs, capsule inputs, eye toggles, dual-role cards, 3D gradient action button, VIP RFID keycards (Zainab Malik & Chef Farhan).
- **Navigation:** Entry point; Customer → Menu, Manager → ManagerDashboard.
- **State:** useForm hook (values, errors, handleChange, handleSubmit), showPassword, isSubmitting, selectedPass.
- **Hooks:** Custom useForm, useRef for 3D animations, useAuth, useTheme.

7.2. Menu Browsing Screen (`MenuScreen.js`)

- **Purpose:** High-end culinary showcase with category filtering, 100% Halal Artisan banner, and special highlights.
- **UI Elements:** Horizontal category chips, pull-to-refresh FlatList, dish cards with PKR price, rating, prep time, Daily Special badge, heart toggle.
- **Navigation:** Default tab in CustomerTabs.
- **State:** menuItems, isLoading, error, refreshing, selectedCategory, favouriteIds.
- **Hooks:** useState, useEffect (1.5s fetch + cleanup), useMemo, useCallback.

7.3. Search and Scroll Controls (`MenuScreen.js`)

- **Purpose:** High-speed item search with debouncing and scroll-to-top controls.
- **UI Elements:** Search input, clear button, recent search chips, floating Back to top button, render counter.
- **Navigation:** Embedded in Menu screen.
- **State:** searchQuery, debouncedQuery (useDebounce), isSearchFocused, recentSearches, showBackToTop.
- **Hooks:** useRef, useDebounce (400ms), useMemo for filter/sort pipeline.

7.4. Profile and Theme Screen (`ProfileScreen.js`)

- **Purpose:** Black-Tie VIP Patron Passport, dining metrics analytics, and luxury theme configuration.
- **UI Elements:** Gold-foiled membership card (Zainab Malik), metric pods (14 visits, Rs 48.5k spent, 2,450 pts), dietary prefs, theme switch, sign out.
- **Navigation:** Fourth tab; logout → Login.
- **State:** From useAuth and useTheme, local dietaryPrefs.
- **Hooks:** useAuth (user details, logout), useTheme (isDark, toggleTheme).

7.5. Cart Screen (`CartScreen.js`)

- **Purpose:** Interactive culinary tray with quantity adjustments, dietary notes, and voucher redemption.
- **UI Elements:** Cart item list with quantity steppers, per-item instructions, delete, promo code entry (AURA10, AURA20, AURA50), checkout.
- **Navigation:** Second tab; checkout → OrderSummary.
- **State:** promoInput, promoError, promoSuccess, items from useCart.
- **Hooks:** useCart (dispatch actions), useTheme for tokens.

7.6. Order Summary Screen (`OrderSummaryScreen.js`)

- **Purpose:** Itemized financial verification and dining mode selection before kitchen dispatch.
- **UI Elements:** Dine-in/Takeaway tabs, table selector/pickup time, itemized breakdown, tax/service summary, confirm button.
- **Navigation:** From CartScreen; on confirm → OrderTracking.
- **State:** diningMode, selectedTable, pickupTime.
- **Hooks:** useMemo (subtotal, 5% service, 15% tax, discount, total), useOrders, useCart.

7.7. Table Reservation Screen (`ReservationScreen.js`)

- **Purpose:** Dine-in table booking portal with real-time slot conflict and capacity validation.
- **UI Elements:** Date picker chips, hourly time slot pills (12:00 to 22:00), party size stepper (1-12), phone input, confirmation modal, My Reservations list.
- **Navigation:** Third tab in CustomerTabs.
- **State:** Delegated to useReservation hook.
- **Hooks:** useReservation (slot selection, booking, cancellation), useTheme.

7.8. Order Tracking & Manager Dashboard (`OrderTrackingScreen.js` & `ManagerDashboardScreen.js`)

- **Purpose:** Real-time kitchen progress monitor and comprehensive manager operations console.
- **UI Elements:** Tracking: Status progress bar, elapsed timer, order receipt. Manager: 3-tab interface (Orders, Reservations, Menu CRUD).
- **Navigation:** OrderTracking from OrderSummary; ManagerDashboard from manager login.
- **State:** activeTab, elapsedSeconds, editModalVisible.
- **Hooks:** useEffect + setInterval for status progression, cleanup functions, useOrders, useMenu, useReservation.